



Lab #8 - Logic Operations & Branching

Logical Instructions:

Instruction	Description	Example	Valid Operand Combinations
AND	Performs a bitwise AND operation between two operands. Only sets bits in the result if both bits are 1.	AND EAX, EBX ; Bitwise AND between EAX and EBX.	AND reg, reg AND reg, imm AND reg, mem
OR	Performs a bitwise OR operation between two operands. Sets a bit if either or both bits are 1.	OR EAX, EBX ; Bitwise OR between EAX and EBX.	OR reg, reg OR reg, imm OR reg, mem
XOR	Performs a bitwise XOR (exclusive OR) operation. Sets a bit if one (but not both) of the bits is 1.	XOR EAX, EBX ; Bitwise XOR between EAX and EBX.	XOR reg, reg XOR reg, imm XOR reg, mem
NOT	Performs a bitwise NOT operation, flipping all bits (1s become 0s and vice versa).	NOT EAX ; Bitwise negation of EAX (inverting all bits).	NOT reg NOT mem
TEST	Performs a bitwise AND operation between two operands, but doesn't modify the destination operand. Updates the flags according to the result, but does not store the result.	TEST EAX, EBX ; Tests the bitwise AND of EAX and EBX.	TEST reg, reg TEST reg, imm TEST reg, mem

Comparison:

The CMP (compare) instruction in x86 assembly is used to compare two values by performing a subtraction between the two operands, but it does not store the result. Instead, it updates the CPU's flags based on the result of the subtraction, which can then be used to make decisions via conditional jumps (like JE, JNE, JL, JLE, JA, JB, etc.). The two operands can be registers, immediate values, or memory locations. The flags that are primarily affected by CMP include the Zero Flag (ZF), Sign Flag (SF), Overflow Flag (OF), and Carry Flag (CF).

The behavior of CMP is the same as the SUB instruction but without storing the result in a destination register. After a CMP, the subsequent instructions that follow can check the flags to determine the relationship between the two values, whether they are equal, one is greater, or one is less.

CMP op1, op2

CMP Instruction

- When two **unsigned operands** are compared, the Zero flag and Carry flag indicate the following relations between operands:

CMP Results	ZF	CF
Destination < source	0	1
Destination > source	0	0
Destination = source	1	0

- When two **signed operands** are compared, the Sign, Zero, and Overflow flags indicate the following relations between operands:

CMP Results	Flags
Destination < source	SF ≠ OF
Destination > source	SF = OF
Destination = source	ZF = 1

18

Figure 1. Course slides - Topic 5 Part 2 - Slide 18

The JMP Instruction and Conditional Jumps in Assembly:

In x86 assembly, the JMP instruction is an unconditional jump that transfers control to a specified label or address. It's useful for implementing loops, skipping sections of code, or transferring control to different parts of a program without evaluating any conditions. In addition to JMP, there are conditional jump instructions like JZ, JNZ, JE, and JNE. These conditional jump instructions allow control flow to branch based on the results of previous instructions, typically following a CMP instruction that sets the appropriate flags. For instance, JZ (Jump if Zero) will make the program jump to a specified label if the Zero Flag (ZF) is set, which indicates that a comparison resulted in equality. These instructions are essential for making decisions and implementing control flow mechanisms like loops and conditionals in assembly programs.

Instruction	Description	Example
JMP	Unconditionally jumps to a specified label or address.	JMP label1
JE	Jumps if Equal (Zero Flag is set).	JE label2
JZ	Jumps if Zero Flag is set (same as JE).	JZ label3
JNE	Jumps if Not Equal (Zero Flag is clear).	JNE label4
JNZ	Jumps if Not Zero (same as JNE).	JNZ label5
JA	Jumps if Above (first operand > second, unsigned).	JA label6
JAЕ	Jumps if Above or Equal (unsigned comparison).	JAЕ label7
JB	Jumps if Below (first operand < second, unsigned).	JB label8
JBE	Jumps if Below or Equal (unsigned comparison).	JBE label9
JG	Jumps if Greater (first operand > second, signed).	JG label10
JGE	Jumps if Greater or Equal (signed comparison).	JGE label11
JL	Jumps if Less (first operand < second, signed).	JL label12
JLE	Jumps if Less or Equal (signed comparison).	JLE label13
JC	Jumps if Carry Flag is set (unsigned overflow).	JC label14
JNC	Jumps if Carry Flag is not set.	JNC label15
JO	Jumps if Overflow Flag is set (signed overflow).	JO label16
JNO	Jumps if Overflow Flag is not set.	JNO label17
JS	Jumps if Sign Flag is set (negative result).	JS label18
JNS	Jumps if Sign Flag is not set (non-negative result).	JNS label19

Example Assembly Code with Jumps

Below is an example of how to use JMP and conditional jumps like JZ and JNZ in assembly:

```
INCLUDE Irvine32.inc
```

```

.data
    num1 DWORD 10
    num2 DWORD 20
    resultEqual BYTE "Numbers are equal", 0
    resultNotEqual BYTE "Numbers are not equal", 0

.code
main PROC
    ; Load numbers into registers
    MOV EAX, num1
    MOV EBX, num2

    ; Compare the numbers
    CMP EAX, EBX

    ; Jump if equal
    JE equal

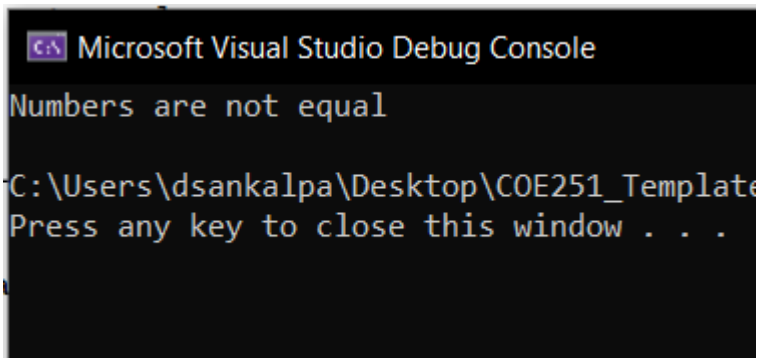
    ; Jump if not equal (this is executed if the JE condition is false)
    JNE not_equal

equal:
    MOV EDX, OFFSET resultEqual
    CALL WriteString
    JMP done

not_equal:
    MOV EDX, OFFSET resultNotEqual
    CALL WriteString

done:
    CALL CrLf
    EXIT
main ENDP
END main

```



Lab Work (Questions 1-3):

1. Write an assembly program that reads a positive integer from the user and calculates the sum of its digits using **DIV**. Then, use the **AND** instruction to check whether the sum is even or odd. Display the sum and a message indicating whether it is even or odd.

```
Microsoft Visual Studio Debug Console
Enter positive number: 1234
The sum of digits is 10 and the sum is even!
C:\Users\dsankalpa\Desktop\COE251_Template\COE251_T
To automatically close the console when debugging s
le when debugging stops.
Press any key to close this window . . .

Microsoft Visual Studio Debug Console
Enter positive number: 52
The sum of digits is 7 and the sum is odd!
C:\Users\dsankalpa\Desktop\COE251_Template\COE
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
prompt BYTE "Enter a positive number: ",0
sum DWORD 0
msg BYTE "The sum of digits is ", 0
even1 BYTE " and the sum is even!",0
odd1 BYTE " and the sum is odd!",0
```

.code

main PROC

```
mov edx, OFFSET prompt
call writestring
```

```
call readInt
```

calc:

```
mov edx, 0
mov ebx, 10
div ebx
add sum, edx
cmp eax, 0
jne calc
```

```
mov edx, OFFSET msg
call writestring
```

```
mov eax, sum
call writedec
```

```
AND eax, 1
cmp eax, 0
jne odd2
mov edx, OFFSET even1
call writestring
jmp done
```

```

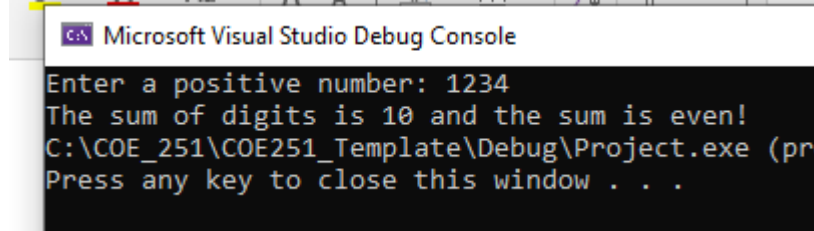
    odd2:
        mov edx, OFFSET odd1
        call writestring

    done:
        exit

main ENDP
END main

```

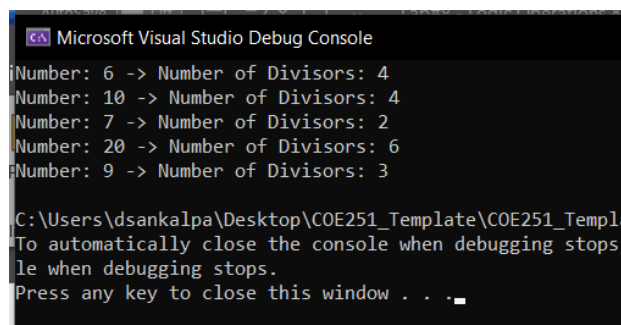
Screenshot:



- Write an assembly program that takes an array of positive integers and, for each element, prints the number of positive divisors it has. A divisor of a number n is any number that divides n without leaving a remainder. For example:
 The number 6 has 4 divisors: 1, 2, 3, 6
 The number 10 has 4 divisors: 1, 2, 5, 10
 The number 7 has 2 divisors: 1 and 7

You may use the following test array: numbers DWORD 6, 10, 7, 20, 9

Important: Your program must be written in a way that it continues to work correctly if the array is replaced with one of any other size (e.g., 3 elements, 10 elements, etc.).



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```

    num BYTE "Number: ",0
    numbers DWORD 6,10,7,20,9
    msg BYTE " -> Number of Divisors: ",0

```

.code

main PROC

```

mov ecx, lengthof numbers
mov esi, 0
first:
    mov edx, OFFSET num
    call writestring

    mov eax, numbers[esi]
    call writedec

    mov edx, OFFSET msg
    call writestring

    push ecx
    mov ecx, numbers[esi]
    mov ebx, 0
    second:
        mov eax, numbers[esi]
        mov edx, 0
        div ecx
        cmp edx, 0
        jne exit1
        inc ebx
    exit1:
        loop second
    pop ecx

    mov eax, ebx
    call writedec

    call crlf
    add esi, 4
loop first
main ENDP
END main

```

Screenshot:

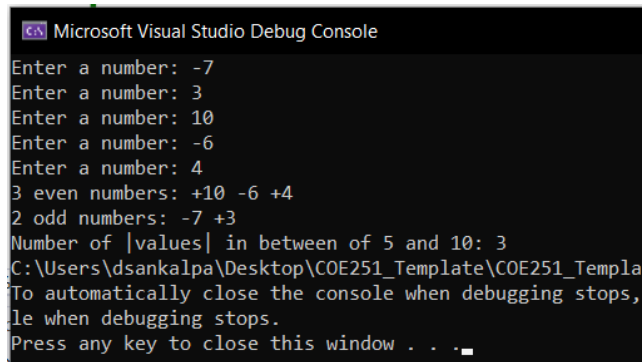
```

Microsoft Visual Studio Debug Console
C:\COE_251\COE251_Template\Debug\Project.exe
Number: 6 -> Number of Divisors: 4
Number: 10 -> Number of Divisors: 4
Number: 7 -> Number of Divisors: 2
Number: 20 -> Number of Divisors: 6
Number: 9 -> Number of Divisors: 3
Press any key to close this window . . .

```

- Write an assembly program that reads 5 signed integers from the user and counts how many of the numbers are even and how many are odd. Use the **IDIV** instruction to determine whether the number is even or odd. The even/odd classification must be done using the original signed value of each number. Additionally, for each number, check whether its absolute value falls in the range [5, 10], inclusive. Use the **NEG** instruction to compute the absolute value if the number is negative.

Display the list of even and odd numbers and the total number of entries whose absolute value lies within the specified range. You may assume that the user will always enter a mix of even and odd numbers, and that at least one number will fall within the range.



The screenshot shows the Microsoft Visual Studio Debug Console with the following text:

```
Enter a number: -7
Enter a number: 3
Enter a number: 10
Enter a number: -6
Enter a number: 4
3 even numbers: +10 -6 +4
2 odd numbers: -7 +3
Number of |values| in between of 5 and 10: 3
C:\Users\dsankalpa\Desktop\C0E251_Template\C0E251_Templa
To automatically close the console when debugging stops,
le when debugging stops.
Press any key to close this window . . .
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
prompt BYTE "Enter a number: ",0
evennum DWORD 0
oddnum DWORD 0
evenarr DWORD 5 DUP(?)
oddarr DWORD 5 DUP(?)
original DWORD 0
count DWORD 0
msg1 BYTE " even numbers: ",0
msg2 BYTE " odd numbers: ",0
abs BYTE "Number of |values| in between of 5 and 10: ",0
```

.code

main PROC

```
mov ecx, 5
mov ebx, 2
mov esi, 0
mov edi, 0
first:
    mov edx, OFFSET prompt
    call writestring

    call readint
    push eax
    cdq
    idiv ebx
    cmp edx, 0
    jne odd1
    pop eax
    mov evenarr[esi], eax
    inc evennum
    add esi, 4
    jmp range_
odd1:
    inc oddnum
```

```

        pop eax
        mov oddarr[edi], eax
        add edi, 4

range_:
        cmp eax, 0
        jge compare
        neg eax
        compare:
            cmp eax, 5
            jb skip
            cmp eax, 10
            ja skip
        inc count
    skip:
    loop first

    mov eax, evennum
    call writeDec

    mov edx, OFFSET msg1
    call writestring

    mov esi, 0
    mov ecx, evennum
    printeven:
        mov eax, evenarr[esi]
        call writeint

        mov al, ' '
        call writechar

        add esi, 4
    loop printeven

    mov eax, oddnum
    call writeDec

    mov edx, OFFSET msg2
    call writestring

    mov esi, 0
    mov ecx, oddnum
    printodd:
        mov eax, oddarr[esi]
        call writeint

        mov al, ' '
        call writechar

        add esi, 4
    loop printodd

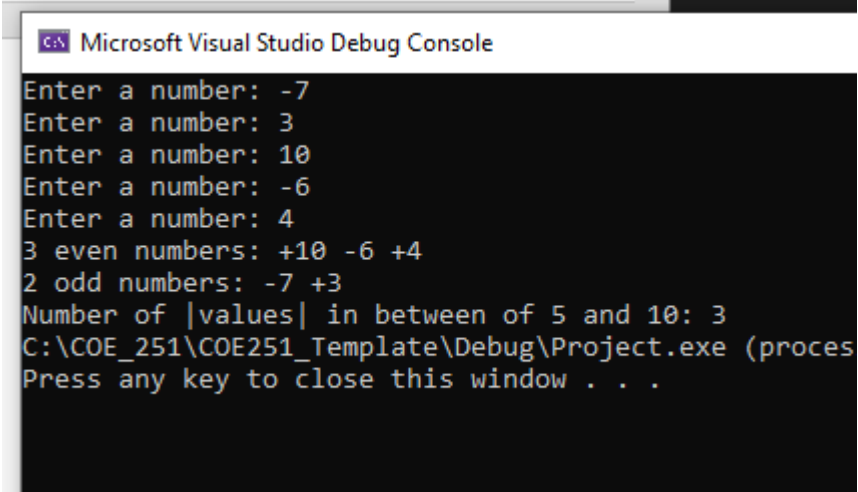
    mov edx, OFFSET abs

```



```
    call writestring  
  
    mov eax, count  
    call writedec  
  
exit  
main ENDP  
END main
```

Screenshot:



The screenshot shows the Microsoft Visual Studio Debug Console window. The title bar reads "C:\ Microsoft Visual Studio Debug Console". The console output is as follows:

```
Enter a number: -7  
Enter a number: 3  
Enter a number: 10  
Enter a number: -6  
Enter a number: 4  
3 even numbers: +10 -6 +4  
2 odd numbers: -7 +3  
Number of |values| in between of 5 and 10: 3  
C:\COE_251\COE251_Template\Debug\Project.exe (proces  
Press any key to close this window . . .
```